

Welcome to the Labs

Candy Catcher



Thank you to our Sponsors!

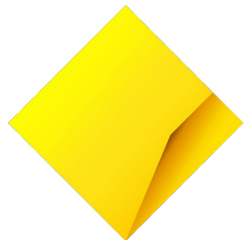
Platinum Sponsors:



Australian Government

Australian Signals Directorate

Gold Sponsor:



**Commonwealth
Bank**

Silver Sponsor:



Who are the tutors?

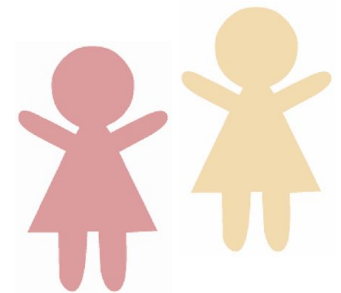
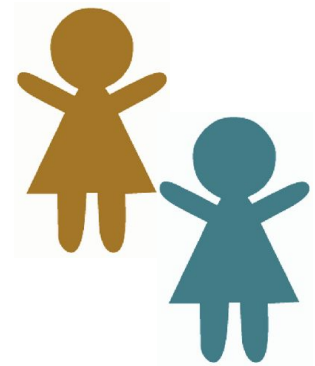


Who are you?



Two Truths and a Lie

1. Get in a group of 3-5 people
2. Tell them three things about yourself:
 - a. Two of these things should be true
 - b. One of these things should be a lie!
3. The other group members have to guess which is the lie



Log on

Log on and jump on the GPN website

girlsprogramming.network/workshop

Click on your node location

Click on your room.

From this page you can see:

- These **slides** (to take a look back or go on ahead).
- A digital copy of your **workbook**.
- Other helpful bits to use through the day!

Tell us you're here!

Click on the
Start of Day Survey
and fill it in now!



Start of Day Survey

Today's project!

Candy Catcher

Candy Catcher

- You're going to build a game called Candy Catcher using Python and Pygame Zero
- Move your hero around the screen with your keyboard to collect as much candy as you can
- Collect all the candy to win!



Introduction to Edstem



Log on

Click on your **Workbook** link to take you into EdStem

Workbook

Slides



Signing up to Edstem

Log in if you already have a an “Edstem” account from a past GPN

Already have an account? [Log in](#)

If you haven't got an account, let's make one:

1. Type in your Full Name
2. Type in your personal email
3. Click Create Account
4. Go to your email and verify your new account
5. Create a password

Full name

Email

you@girlsprogramming.network

Create account

Click Join Course

Join course

The name of your course will be at the top : —————>

Guess Who G

If you don't have access to your email account, ask a tutor for a GPN Edstem login

Getting to the lessons

1. Once you are in the course, you'll be taken to a discussion page.
2. Click the button for the lessons page (top right - looks like a book)



The set up of the workbook

The main page:

1. Heading at the top that tells you the project you are in
2. List of “Chapters” called something like **1: Who are you?**
They have an icon that looks like this: 
3. To complete your project, work through the chapters one at a time



Inside a Chapter

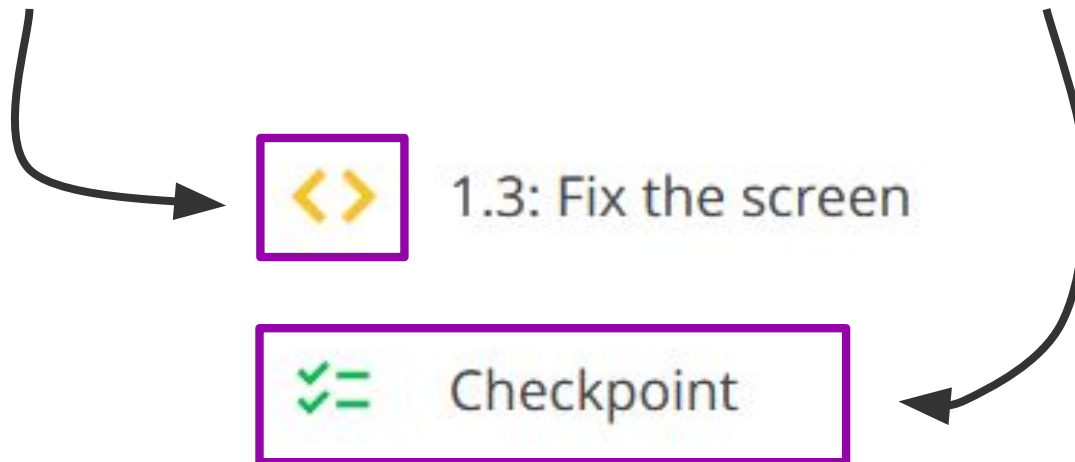
Inside a Chapter there are two main types of pages:

1. Lesson pages

The lessons are where you will do your coding. They have this icon:

2. Checkpoints

Complete the checkpoint to move to the next chapter. Make sure you scroll down to see all the questions listed.

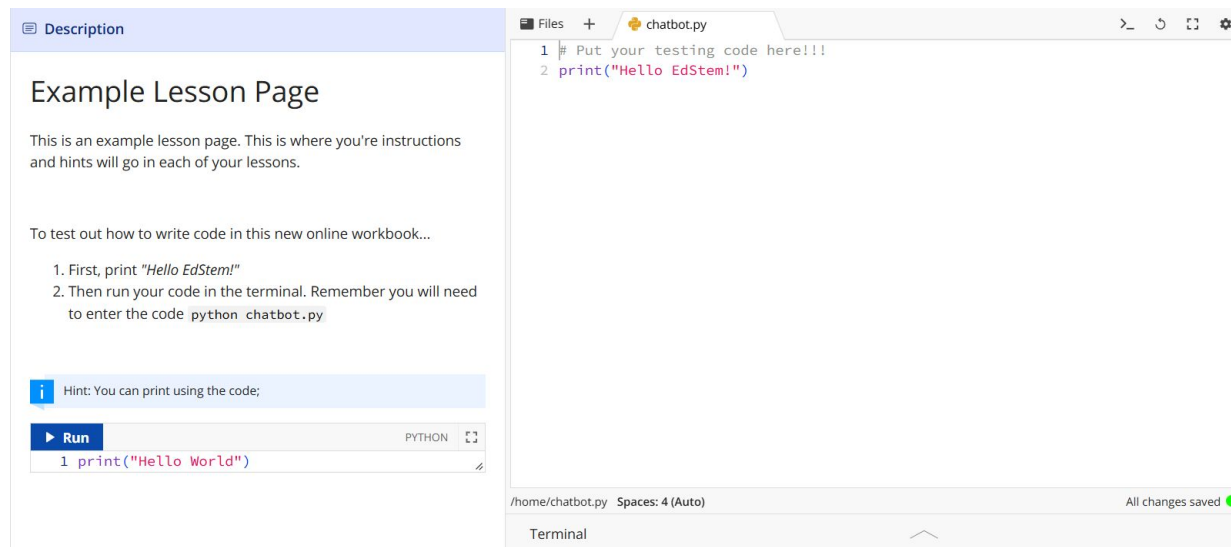


How to do the work

In each lesson there is:

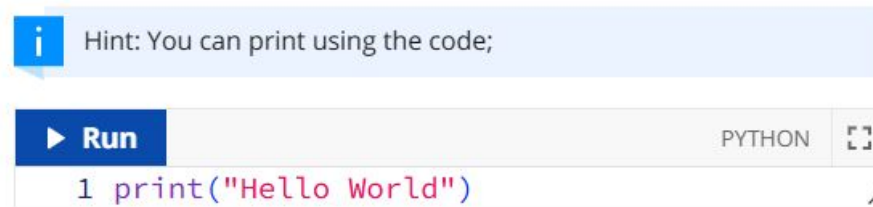
1. A section on left with instructions for that lesson
2. A section on right for your code

You will need to **copy your code from the last lesson**, then follow the instructions to change your code.



Hints

Sometimes in a lesson, there's some code we want you to do that might be a bit tricky, to help you out we've added some hints. They look like this:



If you press the blue run button it will show you what that code does, you can even change the code to see if how it changes.

These are **just hints** make sure you're not copying the hint into your code as it will likely end up breaking. They are just to show you the kinds of things you can do.

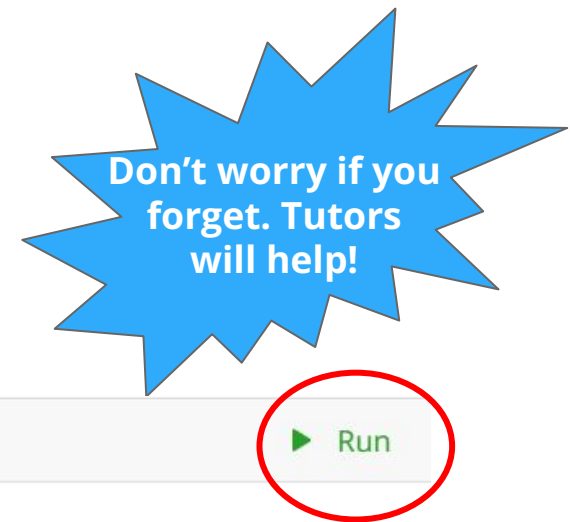
Running your code...

To run your code, click



Your code will run and any output will display in the Console.
You can click the button again to rerun your code.

It should look like this;



Some shortcuts...

There are a couple things you can do to make copying your code from one page to another easier.

- 1) **Ctrl + A** Pressing these keys together will select all the text on a page
- 2) **Ctrl + C** Pressing these keys together will copy anything that's selected
- 3) **Ctrl + V** Pressing these keys together will paste anything you've copied

On Macs use Command (⌘) instead of Ctrl

Intro to Programming



What is programming?



Programming is giving computers a set of instructions!

We use a programming language to talk to computers



People are smart! Computers are dumb!

Programming is like a recipe - a set of instructions to follow

- Works well if the steps are in the right order
- Not so good if out of order



Computers will do EXACTLY what you tell them to do

They're not good at filling in gaps or knowing if something's wrong



Everyone/thing has strengths!



- Understand instructions despite:
 - Spelling mistakes
 - Typos
 - Confusing parts
- Solve problems
- Tell computers what to do
- Get smarter every day



- Does exactly what you tell it
- Does it the same every time
- Doesn't need to sleep!
- Will work for hours on end!
- Get smarter when you tell them how

Mistakes are great!

*SyntaxError:
Invalid Syntax*

Good work you made an error!

*ImportError
No module
named humour*

- Programmers make A LOT of errors!
- Errors give us hints to find mistakes
- Run your code often to get the hints!!
- Mistakes won't break computers!



*KeyError:
'Hairy Potter'*

*AttributeError:
'NoneType' object
has no attribute
'foo'*

*TypeError: Can't
convert 'int' object
to str implicitly*



We can learn from our mistakes!

Error messages help us fix our mistakes!
We read error messages from bottom to top

Traceback (most recent call last):

File "C:/Users/Madeleine/Desktop/tmp.py", line 9, in <module>
 print("I have " + 5 + " apples")

TypeError: can only concatenate str (not "int") to str

1. What went wrong

2. What code didn't work

3. Where that code is



Adding a comment!

Sometimes we want to write things in our file that the computer doesn't look at. We can use **comments** for that!

Sometimes we want to write a note for a people to read

```
# This code was written by Vivian
```

And sometimes we want to not run some code (but don't want to delete it!)

```
# print("Goodbye world!")
```

Lots of comments to help you ...

Your workbook today has a code template.

It has lots of comments to help you know where to put your code

Your workbook will ask you to find a particular comment and then add your code under it.



```
Files + candy_catcher.py
1 # <insert your name here>
2
3 # imports
4
5 # create constants
6
7 # print welcome
8
9 # make hero
10
11 # make candy
12
13 # add candy
14
15 # add game bits
16
```

Write some code!!

This is the first bit of code we will do.

```
print('hello world')
```

It prints the words “hello world” on the screen!

Tell me more!

We can `print` on many lines at once using `"""`!

```
>>> print("""Hello world.
```

```
This is me!
```

```
Life should be fun for everyone""")
```

Tell me more!

We can `print` on many lines at once using `"""`

```
print("""Hello world.
```

```
This is me!
```

```
Life should be fun for everyone""")
```

```
Hello world.
```

```
This is me!
```

```
Life should be fun for everyone
```

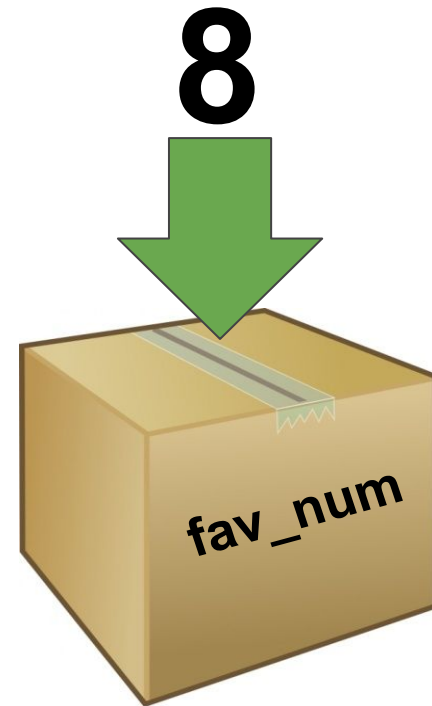
Variables



No Storing is Boring!

It's useful to be able to remember things for later!
Computers remember things in "**variables**"

Variables are like putting things into a **labeled cardboard box**.



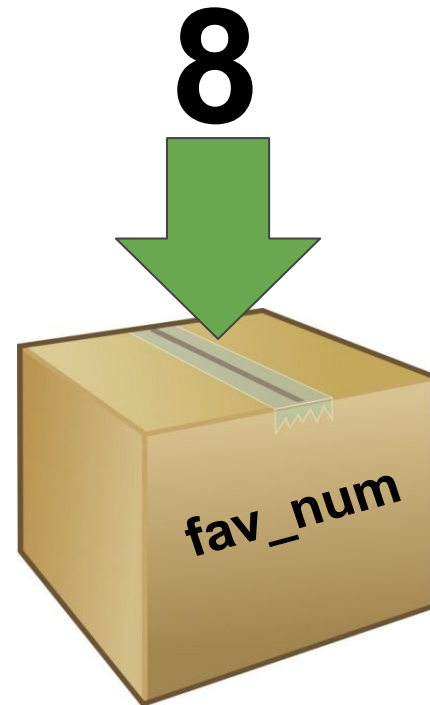
No Storing is Boring!

It's useful to be able to remember things for later!
Computers remember things in "**variables**"

**Let's make our favourite
number 8 today!**

In our code we create a variable
and set it to a value like this

`fav_num = 8`



Variables

Instead of writing the number 8, we can write fav_num. The computer will find the latest value of fav_num and use it.

fav_num = 8



fav_num - 6

=>

fav_num + 21

=>

fav_num * 2

=>

fav_num / 2

=>

Variables

Instead of writing the number 8, we can write fav_num.

fav_num = 8



$$\text{fav_num} - 6 \\ \Rightarrow 2$$

$$\text{fav_num} + 21 \\ \Rightarrow$$

$$\text{fav_num} * 2 \\ \Rightarrow$$

$$\text{fav_num} / 2 \\ \Rightarrow$$

Variables

Instead of writing the number 8, we can write fav_num.

fav_num = 8



$$\text{fav_num} - 6 \\ \Rightarrow 2$$

$$\text{fav_num} + 21 \\ \Rightarrow 29$$

$$\text{fav_num} * 2 \\ \Rightarrow$$

$$\text{fav_num} / 2 \\ \Rightarrow$$

Variables

Instead of writing the number 8, we can write fav_num.

fav_num = 8



$$\text{fav_num} - 6 \\ \Rightarrow 2$$

$$\text{fav_num} + 21 \\ \Rightarrow 29$$

$$\text{fav_num} * 2 \\ \Rightarrow 16$$

$$\text{fav_num} / 2 \\ \Rightarrow$$

Variables

Instead of writing the number 8, we can write fav_num.

fav_num = 8



$$\text{fav_num} - 6 \\ \Rightarrow 2$$

$$\text{fav_num} + 21 \\ \Rightarrow 29$$

$$\text{fav_num} * 2 \\ \Rightarrow 16$$

$$\text{fav_num} / 2 \\ \Rightarrow 4$$

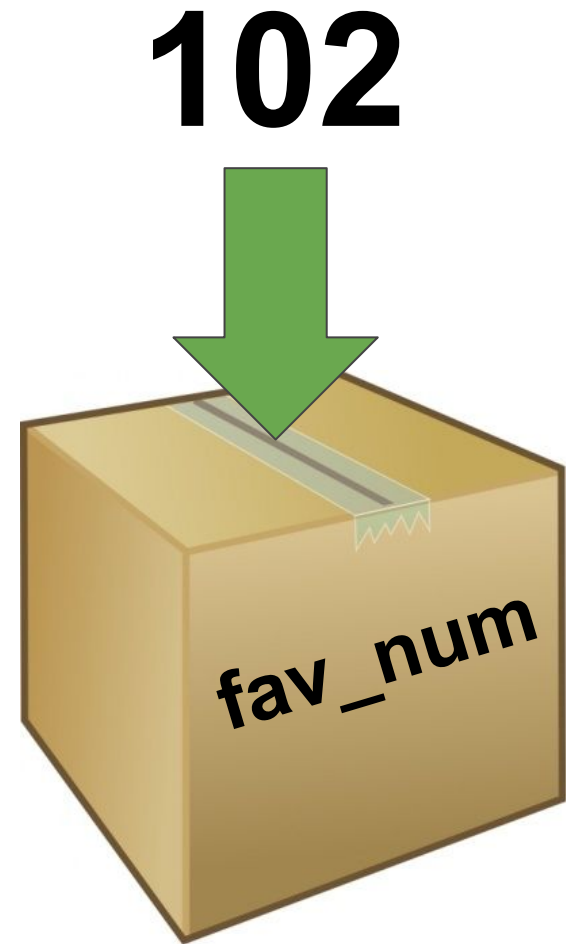
Variables

Variables are useful for storing things that change

(i.e. things that "vary" - hence the word "variable")

Try changing fav_num to **102**

fav_num = 102



Variables

We're able to use our code for a new purpose, without rewriting everything:

`fav_num = 102`



`fav_num - 6`

`=>`

`fav_num + 21`

`=>`

`fav_num * 2?`

`=>`

`fav_num / 2?`

`=>`

Variables

We're able to use our code for a new purpose, without rewriting everything:

`fav_num = 102`



`fav_num - 6`
=> 96

`fav_num + 21`
=>

`fav_num * 2?`
=>

`fav_num / 2?`
=>

Variables

We're able to use our code for a new purpose, without rewriting everything:

`fav_num = 102`



`fav_num - 6`
=> 96

`fav_num + 21`
=> 123

`fav_num * 2?`
=>

`fav_num / 2?`
=>

Variables

We're able to use our code for a new purpose, without rewriting everything:

`fav_num = 102`



`fav_num - 6`
=> 96

`fav_num + 21`
=> 123

`fav_num * 2?`
=> 204

`fav_num / 2?`
=>

Variables

We're able to use our code for a new purpose, without rewriting everything:

`fav_num = 102`



`fav_num - 6`
=> 96

`fav_num + 21`
=> 123

`fav_num * 2?`
=> 204

`fav_num / 2?`
=> 51

Reusing variables

We can replace values in variables:

```
animal = "dog"
print("My favourite animal is a " + animal)
animal = "cat"
print("My favourite animal is a " + animal)
animal = animal + "dog"
print("My favourite animal is a " + animal)
```

What will this output?

Reusing variables

We can replace values in variables:

```
animal = "dog"
print("My favourite animal is a " + animal)
animal = "cat"
print("My favourite animal is a " + animal)
animal = animal + "dog"
print("My favourite animal is a " + animal)
```

```
My favourite animal is a dog
```

Reusing variables

We can replace values in variables:

```
animal = "dog"
print("My favourite animal is a " + animal)
animal = "cat"
print("My favourite animal is a " + animal)
animal = animal + "dog"
print("My favourite animal is a " + animal)
```

What will this output?

Reusing variables

We can replace values in variables:

```
animal = "dog"
print("My favourite animal is a " + animal)
animal = "cat"
print("My favourite animal is a " + animal)
animal = animal + "dog"
print("My favourite animal is a " + animal)
```

```
My favourite animal is a dog
My favourite animal is a cat
```

Reusing variables

We can replace values in variables:

```
animal = "dog"
print("My favourite animal is a " + animal)
animal = "cat"
print("My favourite animal is a " + animal)
animal = animal + "dog"
print("My favourite animal is a " + animal)
```

What will this output?

Reusing variables

We can replace values in variables:

```
animal = "dog"
print("My favourite animal is a " + animal)
animal = "cat"
print("My favourite animal is a " + animal)
animal = animal + "dog"
print("My favourite animal is a " + animal)
```

```
My favourite animal is a dog
My favourite animal is a cat
My favourite animal is a catdog
```

Different data!

There are lots of types of data!

Our main ones for today are:

Strings

Things in quotes used for storing text

`"This is a string"`

Ints

Whole numbers we can do maths with

```
a = 1
b = 2
print(a + b)
```

Global variables

This is how we can make shared, global variables:

```
def add():  
    global total  
    total = total + 1  
  
total = 5  
add()  
print(total)
```

Intro to PyGame Zero

Making it into a game!



What is Pygame Zero?

We use pygame zero to allow our code to do some cool things.



Pygame Zero Setup

The first thing we need to do to use pygame zero is to write this at the top of your file

```
>>> import pgzrun
```

PyGame Zero Setup

The first thing we need to do to use PyGame Zero is to write this at the top of your file

```
import pgzrun
```

Now to make sure PyGame Zero runs our code we also need another line at the end of our code

```
pgzrun.go()
```

Some Pygame Zero basics

Here's some of the basics of Pygame Zero that you'll need for your game.

Screen:

A game screen will pop up in a window when you run your code.

You can create a screen by setting its size dimensions using keywords WIDTH and HEIGHT

- To make a screen 100 pixels wide x 200 pixels high you would code

```
WIDTH = 100
```

```
HEIGHT = 200
```

You must click on the screen before you start to play your game.



Project time!

You now know all about the basics about
PyGame Zero!

Let's put what we learnt into our project
Try to do Part 1

The tutors will be around to help!

PyGame Zero images

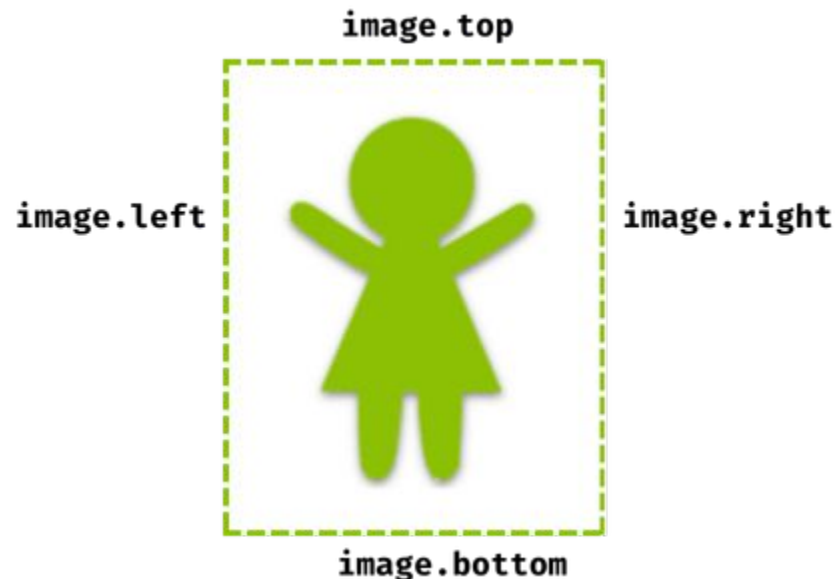


Adding things to our screen!

Images in Pygame zero

Images in Pygame zero are called **Actors**

This is because you can make them move around and do things like actors in a play. Pygame zero stores some information about each of the actors in our game like their position on the screen and what image the actor is.

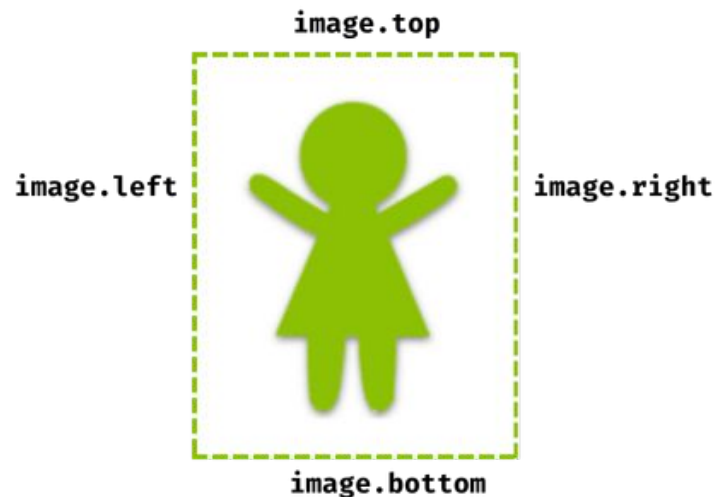


How to make an actor

To make a new actor and tell Pygame zero what image it is you need to write the code:

```
>>> myActor = Actor("myImage")
```

Here the name of our actor is **myActor** and if we need to change anything about it we have to use its name



How to make an actor

To make a new actor and tell Pygame zero what image it is you need to write the code:

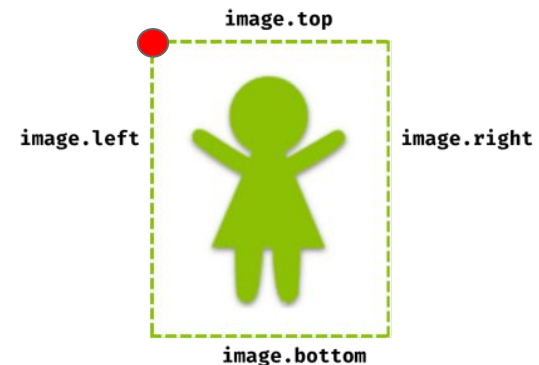
```
myActor = Actor("myImage")
```

Here the name of our actor is **myActor** and if we need to change anything about it we have to use its name

To set our actor's position you use the code:

```
myActor.top = 50
```

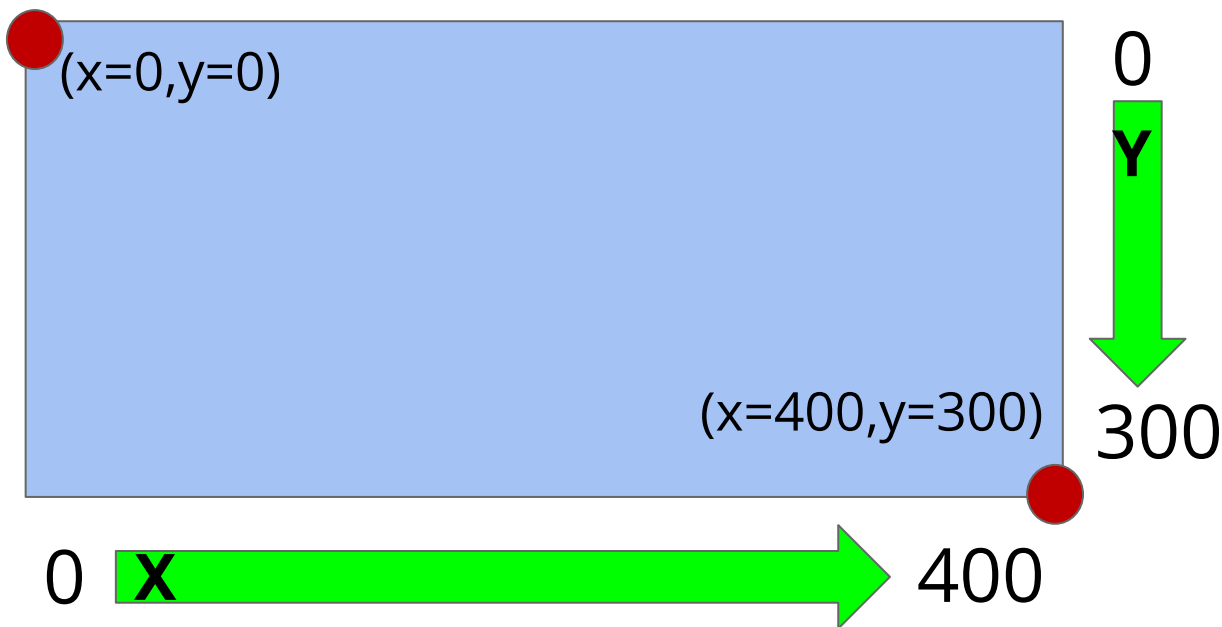
```
myActor.left = 50
```



- Sets top left hand corner of image 50 pixels down from the top of the screen and 50 pixels away from the left of the screen

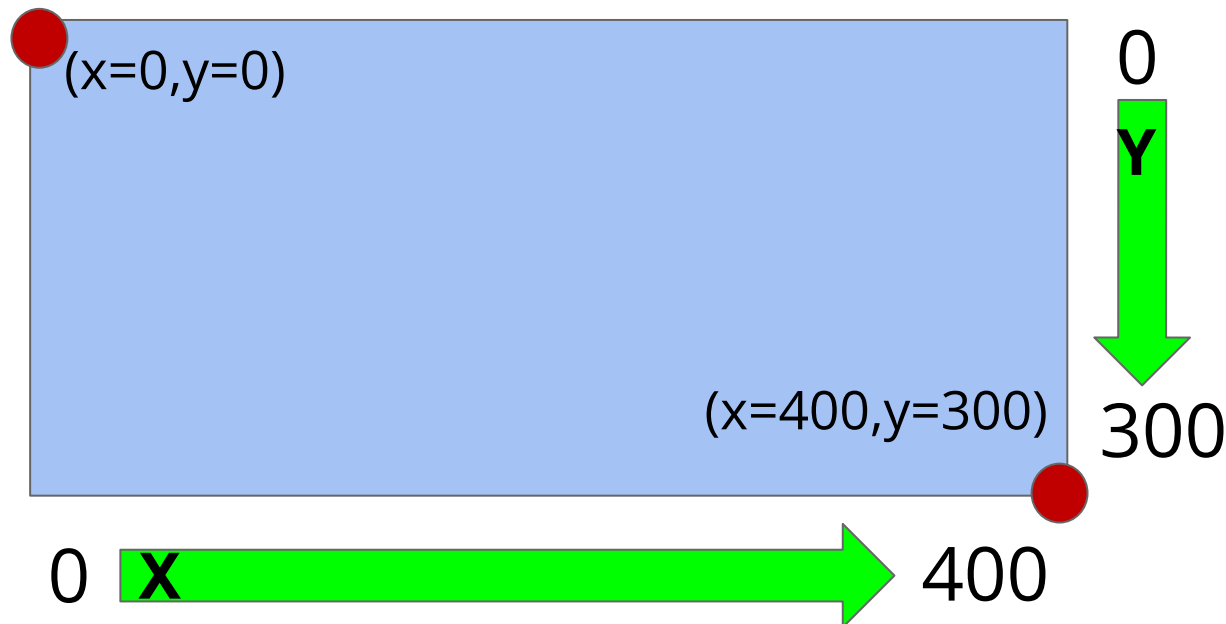
What does the screen look like?

- **left/right** properties increase from left to right (like normal)
- **top/bottom** properties **increase from top to bottom (not normal)**



How to position an actor on the screen

- Change the actor's top/bottom/left/right to move them around the screen
- To move left, decrease left. To move right, increase right.
- To move up, **decrease top**. To move down, **increase bottom**.



Functions

- Sometimes in code we make a **function**.
- A **function** is a set of code that we want to run over and over again.
- It's like a recipe - Instead of writing the same instructions over and over again, you write them once, give them a name, and use that whenever you want to bake!
- **Functions** help us organise our code and make it easier to understand

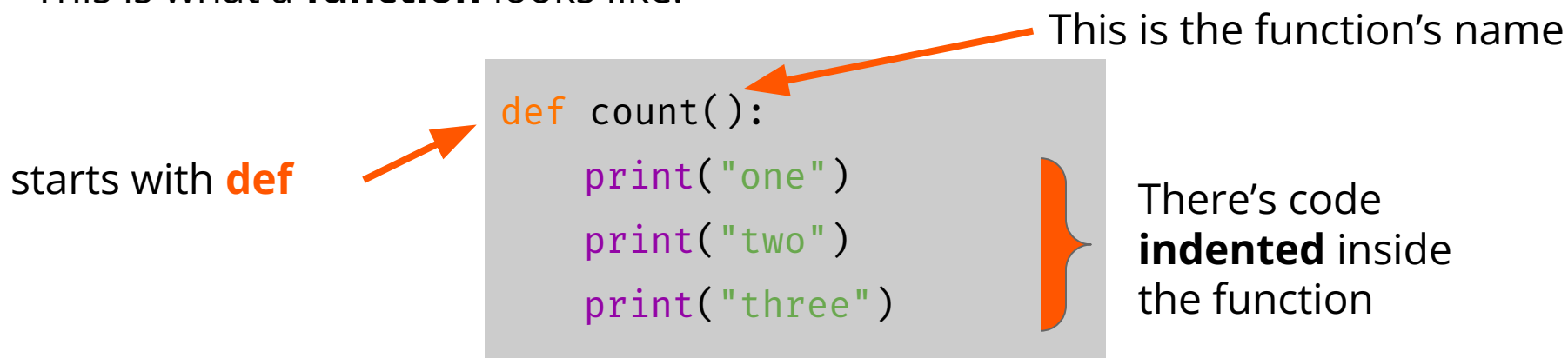
This is what a **function** looks like:

starts with **def**

```
def count():  
    print("one")  
    print("two")  
    print("three")
```

This is the function's name

There's code **indented** inside the function

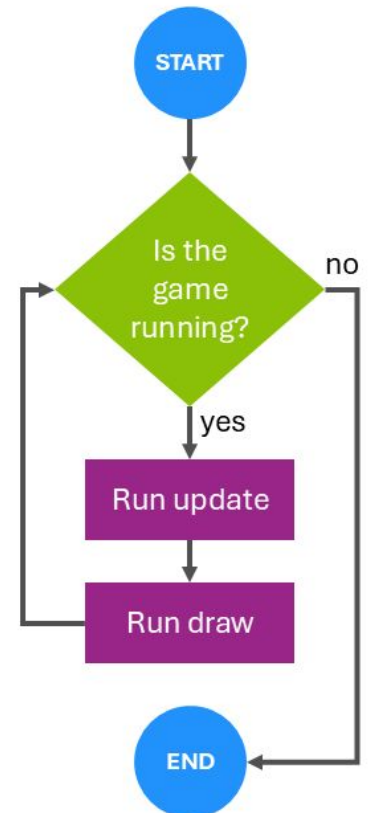
A diagram illustrating the structure of a Python function. It shows a code block with the following text: `def count():` on the first line, followed by three indented lines: `print("one")`, `print("two")`, and `print("three")`. An orange arrow points from the text "starts with def" to the `def` keyword. Another orange arrow points from the text "This is the function's name" to the `count()` part of the first line. A large orange curly brace on the right side of the indented lines points to the text "There's code indented inside the function".

PyGame Zero Functions

- PyGame Zero needs two functions: **update** and **draw**
- It runs these functions over and over.
- You'll see these in your code template.
- Your workbook will ask you to find a particular function and then add your code under it.

```
def update():  
    # This function is to change things every frame
```

```
def draw():  
    # This function is to add things to the screen every frame
```



Other things we'll use ...

We can repaint the background screen of the game to clear it using

screen.fill("blue")

We can check if a particular key was pressed like this

if key == keys.ENTER: is TRUE if ENTER key was pressed

if key == keys.DOWN: is TRUE if DOWN key was pressed

We can check if one Actor collides with another Actor like this

if Actor1.colliderect(Actor2): TRUE if the actors have collided

We can write text onto the screen at position x across and y down using

screen.draw.text ("Hello", (x,y))

Project time!

You now know all about images on Pygame!

See if you can draww part 2!

The tutors will be around to help!

If Statements



Conditions!

Conditions let us make decision.

First we test if the condition is met!

Then maybe we'll do the thing



If it's raining take an umbrella

Yep it's raining

..... take an umbrella

Booleans (True and False)

Computers store whether a condition is met in the form of

True and **False**

To figure out if something is **True** or **False** we do a comparison

`5 < 10`

`3 + 2 == 5`

`5 != 5`

`"Dog" == "dog"`

`"D" in "Dog"`

`"Q" not in "Cat"`

Booleans (True and False)

Computers store whether a condition is met in the form of

True and **False**

To figure out if something is **True** or **False** we do a comparison

<code>5 < 10</code>	True	<code>"Dog" == "dog"</code>
<code>3 + 2 == 5</code>		<code>"D" in "Dog"</code>
<code>5 != 5</code>		<code>"Q" not in "Cat"</code>

Booleans (True and False)

Computers store whether a condition is met in the form of

True and **False**

To figure out if something is **True** or **False** we do a comparison

`5 < 10` **True**

`3 + 2 == 5` **True**

`5 != 5`

`"Dog" == "dog"`

`"D" in "Dog"`

`"Q" not in "Cat"`

Booleans (True and False)

computers store whether a condition is met in the form of

True and **False**

To figure out if something is **True** or **False** we do a comparison

<code>5 < 10</code>	<code>True</code>	<code>"Dog" == "dog"</code>
<code>3 + 2 == 5</code>	<code>True</code>	<code>"D" in "Dog"</code>
<code>5 != 5</code>	<code>False</code>	<code>"Q" not in "Cat"</code>

Booleans (True and False)

Computers store whether a condition is met in the form of

True and **False**

To figure out if something is **True** or **False** we do a comparison

<code>5 < 10</code>	<code>True</code>	<code>"Dog" == "dog"</code>	<code>False</code>
<code>3 + 2 == 5</code>	<code>True</code>	<code>"D" in "Dog"</code>	
<code>5 != 5</code>	<code>False</code>	<code>"Q" not in "Cat"</code>	

Booleans (True and False)

Computers store whether a condition is met in the form of

True and **False**

To figure out if something is **True** or **False** we do a comparison

<code>5 < 10</code>	<code>True</code>	<code>"Dog" == "dog"</code>	<code>False</code>
<code>3 + 2 == 5</code>	<code>True</code>	<code>"D" in "Dog"</code>	<code>True</code>
<code>5 != 5</code>	<code>False</code>	<code>"Q" not in "Cat"</code>	

Booleans (True and False)

Computers store whether a condition is met in the form of

True and **False**

To figure out if something is **True** or **False** we do a comparison

<code>5 < 10</code>	<code>True</code>	<code>"Dog" == "dog"</code>	<code>False</code>
<code>3 + 2 == 5</code>	<code>True</code>	<code>"D" in "Dog"</code>	<code>True</code>
<code>5 != 5</code>	<code>False</code>	<code>"Q" not in "Cat"</code>	<code>True</code>

Conditions

So to know whether to do something, they find out if it's **True**!

```
fave_num = 5
if fave_num < 10:
    print("that's a small number")
```

Conditions

So to know whether to do something, they find out if it's **True**!

```
fave_num = 5
if fave_num < 10:
    print("that's a small number")
```

That's the
condition!

Conditions

So to know whether to do something, they find out if it's **True**!

```
fave_num = 5
if fave_num < 10:
    print("that's a small number")
```

That's the
condition!

Is it **True** that fave_num is less than 10?

- Well, fave_num is 5
- And it's **True** that 5 is less than 10
- So it is **True**!

Conditions

So to know whether to do something, they find out if it's **True**!

```
fave_num = 5
if True:
    print("that's a small number")
```

Put in the
answer to
the question

Is it **True** that fave_num is less than 10?

- Well, fave_num is 5
- And it's **True** that 5 is less than 10
- So it is **True**!

Conditions

So to know whether to do something, they find out if it's **True**!

```
fave_num = 5
if True:
    print("that's a small number")
```

What do you think happens?

Conditions

So to know whether to do something, they find out if it's **True**!

```
fave_num = 5
if True:
    print("that's a small number")
```

What do you think happens?
that's a small number

Conditions

How about a different number???

```
fave_num = 9000  
if fave_num < 10:  
    print("that's a small number")
```



Conditions

Find out if it's **True**!

```
fave_num = 9000  
if False:  
    print("that's a small number")
```

Put in the
answer to
the question

Is it **True** that fave_num is less than 10?

- Well, fave_num is 9000
- And it's not **True** that 9000 is less than 10
- So it is **False**!

Conditions

How about a different number???

```
fave_num = 9000  
if fave_num < 10:  
    print("that's a small number")
```



What do you think happens?

Conditions

How about a different number???

```
fave_num = 9000  
if fave_num < 10:  
    print("that's a small number")
```



What do you think happens?



Nothing!

If statements

```
fave_num = 5  
if fave_num < 10:  
    print("that's a small number")
```

This line ...

... controls this line

If statements

Actually

```
fave_num = 5
if fave_num < 10:
    print("that's a small number")
    print("and I like that")
    print("A LOT!!")
```

This line ...



... controls anything below it
that is indented like this!

If statements

```
fave_num = 5
if fave_num < 10:
    print("that's a small number")
    print("and I like that")
    print("A LOT!!")
```

What do you think happens?

If statements

```
fave_num = 5
if fave_num < 10:
    print("that's a small number")
    print("and I like that")
    print("A LOT!!")
```

```
that's a small number
and I like that
A LOT!!
```

If statements

```
word = "GPN"  
if word == "GPN":  
    print("GPN is awesome!")
```

What happens?

If statements

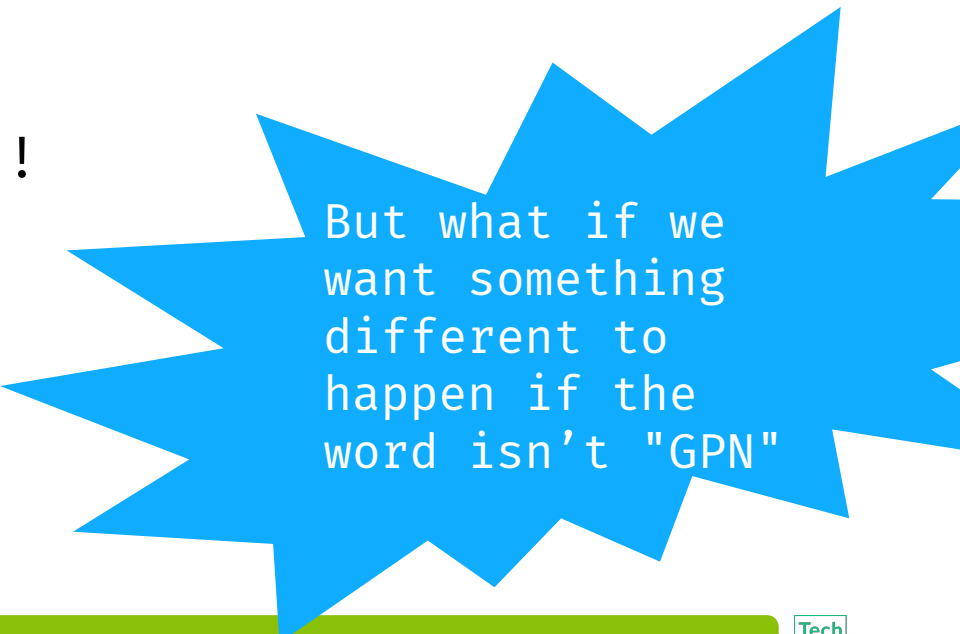
```
word = "GPN"  
if word == "GPN":  
    print("GPN is awesome!")
```

What happens?
GPN is awesome!

If statements

```
word = "GPN"  
if word == "GPN":  
    print("GPN is awesome!")
```

What happens?
GPN is awesome!



But what if we
want something
different to
happen if the
word isn't "GPN"

Else statements

else
statements means
something still
happens if the **if**
statement was
False

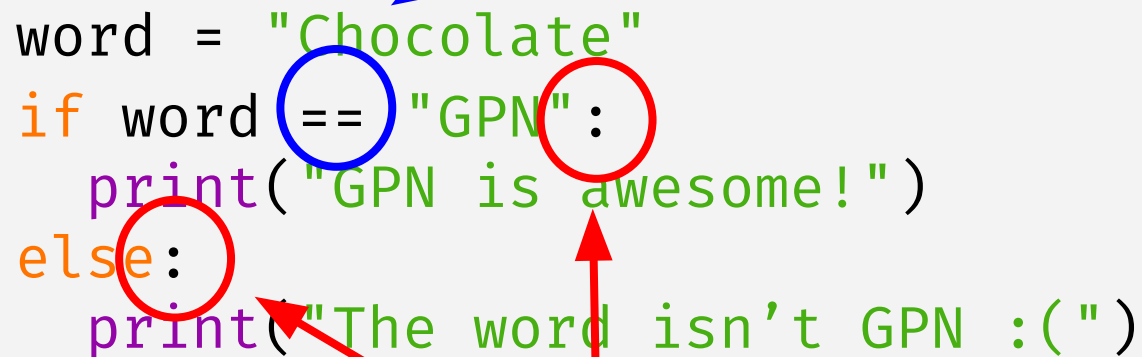
```
word = "Chocolate"  
if word == "GPN":  
    print("GPN is awesome!")  
else:  
    print("The word isn't GPN :(")
```

What happens?
The word isn't GPN :(

== AND : in If/Else Statements

Remember ...

== When testing for equals in your condition



```
word = "Chocolate"
if word == "GPN":
    print("GPN is awesome!")
else:
    print("The word isn't GPN :(")
```

The code snippet is annotated with circles and arrows. A blue circle highlights the '==' operator in the 'if' condition, with a blue arrow pointing to it from the text '== When testing for equals in your condition'. A red circle highlights the colon ':' at the end of the 'if' line, with a red arrow pointing to it from the text 'At end of each if line to say you have finished writing your condition'. Another red circle highlights the colon ':' at the end of the 'else' line, with a red arrow pointing to it from the same text.

•
• **At end of each if line to say you have finished writing your condition**

Project time!

You now know all about **if**!

See **if you can do part 3!**

The tutors will be around to help!

Random!

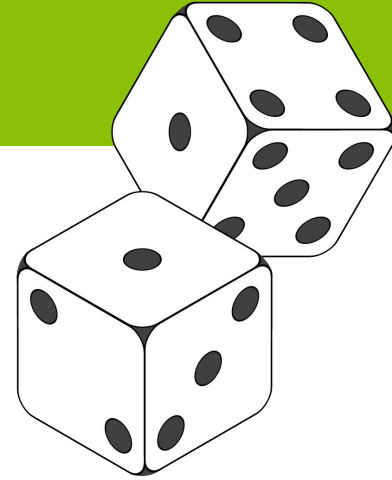


That's so random!

There's lots of things in life that are up to chance or random!



Python lets us **import** common bits of code people use! We're going to use the **random** module!



We want the computer to be random sometimes!



Using the random module

Let's choose a random number.
It's like picking a lottery number.



1. Import the random module at the top of your code like this

```
import random
```

2. Choose a random number within a range using code like this

```
random.randint(0,11)
```

It chooses a random number from the first number up to, but not including, the second!

This would choose 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 or 10

Using the random module

You can also assign your random choice to a variable

```
import random  
lottery_num = random.randint(0,1000)  
print(lottery_num)
```

This would choose a number between 0 and 999



Project Time!

Raaaaaaaaaandom! Can you handle that?

Let's try use it in our project!
Try to do part 4 & 5

The tutors will be around to help!